A PROJECT REPORT

ON

JAVA Project Based Learning

Hack Agon

Submitted in partial fulfillment of the requirement for the award of Degree of Bachelor of Technology in Computer Science & Engineering

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted By:

Akshat Rathore (24EARCS014)

Hemant Singh Tanwar (24EARCS058)

Harsh Sharma (24EARCS055)

Hiten Sharma (24EARCS062)

Arjun Suman (24EARCS028)

Session: 2025-26

Department of Computer Science & Engineering

ARYA COLLEGE OF ENGINEERING & I.T.

SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Computer Science & Engineering

CERTIFICATE OF APPROVAL

The Project Report entitled "**Hack Agon**" submitted by "Akshat Rathore (24EARCS014), Hemant Singh Tanwar (24EARCS058), Harsh Sharma (24EARCS055), Hiten Sharma (24EARCS062), Arjun Suman (24EARCS028)" has been examined by us and is hereby approved for carrying out the project leading to the award of degree "**Bachelor of Technology in Computer Science & Engineering**". By this approval the undersigned does not necessarily endorse or approve any statement made, opinion expressed or conclusion drawn therein, but approve the pursuance of project only for the above mentioned purpose.

Mentor
ER. RamBabu Buri
Professor

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the work embodies in this Project entitled "**Hack Agon**" being submitted by "**Akshat Rathore (24EARCS014), Hemant Singh Tanwar (24EARCS058), Harsh Sharma (24EARCS055), Hiten Sharma (24EARCS062) and Arjun Suman (24EARCS028)**" in partial fulfillment of the requirement for the award of "**Bachelor of Technology in Computer Science & Engineering**" to Rajasthan Technical University, Kota (Raj.) during the academic year 2025-26 is a record of bonafide piece of work, carried out by them under our supervision and guidance in the "**Department of Computer Science & Engineering**", Arya College of Engineering & Information Technology, Jaipur.

Mentor
ER. RamBabu Buri
Professor

HOD
Dr. Akhil Pandey
Professor

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Computer Science & Engineering

DECLARATION

We "**Akshat Rathore, Hemant Singh Tanwar, Harsh Sharma, Hiten Sharma and Arjun Suman**", students of Bachelor of Technology in Computer Science & Engineering, session 2025-26, Arya College of Engineering & Information Technology, Jaipur, here by declare that the work presented in this Project entitled "**Hack Agon**" is the outcome of our own work, is bonafide and correct to the best of our knowledge and this work has been carried out taking care of Engineering Ethics. The work presented does not infringe any patented work and has not been submitted to any other University or anywhere else for the award of any degree or any professional diploma.

Akshat Rathore (24EARCS014)
Hemant Singh Tanwar (24EARCS058)
Harsh Sharma (24EARCS055)
Hiten Sharma (24EARCS062)
Arjun Suman (24EARCS028)

Date : dd-mm-yyyy

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who helped us complete this project. First of all we thank our project guide for constant direction, patient review and helpful suggestions throughout the project. We are grateful to the Head of the Department of Computer Science and Engineering and to the faculty of Arya College of Engineering and IT, Jaipur, for providing the learning environment and the laboratory facilities.

We also thank the open-source community. HackAgon stands on Java, Spring Boot, MySQL, TiDB Cloud, Render, GitHub and many small libraries, all of which are free to use and well documented. The free books listed in the library of the application, written by authors who chose to share their work, were a daily reminder of why this project matters.

Finally we thank our friends and families for their encouragement, and our classmates who tested early versions of the application and told us honestly what was confusing. Working together as a team of five, with each person owning clear modules, taught us as much about collaboration as about code.

Hiten Sharma, Harsh Sharma, Hemanth Singh Tanwar, Arjun Suman and Akshat Rathore

Team Responsibilities

The five members divided the work according to the team plan. The table below shows who owned which modules; Chapter 6 describes each module in detail.

Member	Modules owned
Hiten Sharma	Accounts and OTP; Rooms and matches
Harsh Sharma	Code judge and question bank; Admin and moderation
Hemanth Singh Tanwar	Tournaments and money; Platform core and DevOps
Arjun Suman	Proctoring and host dashboard; Duels and teams
Akshat Rathore	UI and design system; Learning, achievements and AI

We are grateful to our mentor, ER. RamBabu Buri, Professor, and to the Head of the Department, Dr. Akhil Pandey, Professor, whose guidance and encouragement helped us through every stage of the project.

ABSTRACT

HackAgon is a web application for hosting and joining online coding competitions. The problem it addresses is that ordinary hosts such as college clubs, classes and groups of friends have no simple way to run a fair, proctored and optionally paid contest, while players have no easy way to trust that the prize money is safe and the result is honest.

The system is built with Java and Spring Boot on the server, a MySQL-compatible database hosted on TiDB Cloud, and fifteen HTML pages styled with plain CSS and JavaScript. It offers solo, duo, squad and duel rooms; free and paid tournaments with escrow, a published prize split and a flat 1% platform fee on the prize pool; automatic judging against hidden test cases with points, a time bonus and tie-break rules; proctoring through a camera gate, webcam snapshots and a server-side tab-lock; a mistake book, a library of free books, achievements and an AI coach; and an administrator back office.

Accounts are verified with a six-digit one-time password sent by e-mail and stored only as a bcrypt hash. Sessions use signed JWTs. Money is stored as integer paise and every movement is written to an append-only ledger. The database has twenty tables, and the server exposes 76 REST endpoints.

An automated smoke test and a proctoring test were run against a local MariaDB instance; 20 of the 21 planned checks passed in the final runs, and the remaining checks need an AI-coach key that was not configured in the test environment. JavaScript submissions are really executed, while other languages, the payment gateway and SMS delivery are simulated or planned, and the report says so plainly. The application is deployed from GitHub to Render with the database on TiDB Cloud.

Keywords: online coding contest, escrow, proctoring, Java, Spring Boot, MySQL, TiDB Cloud, JWT, OTP.

Project at a Glance

Item	Detail
Platform	Web application for hosting and joining online coding competitions
Back end	Java 17 and Spring Boot 3.2, built with Maven
Database	MySQL-compatible; TiDB Cloud in production, MariaDB 10.11 for local tests
Front end	Fifteen HTML pages with plain CSS and JavaScript
Size	76 REST endpoints, eight controller classes, twenty tables
Deployment	GitHub repository to a Render web service
Duration	6 July 2026 to 6 October 2026
Team	Five members, ten modules

INDEX

1. INTRODUCTION.....1
1.1 Background.....1
1.2 Problem Statement.....1
1.3 Objectives.....1
1.4 Scope of the Project.....2
1.5 Organisation of the Report.....2
2. LITERATURE SURVEY.....3
2.1 Online Judges and Contest Platforms.....3
2.2 Proctoring Approaches.....3
2.3 Money, Escrow and Fairness.....3
2.4 Gap Analysis.....4
3. REQUIREMENT ANALYSIS.....5
4. SYSTEM DESIGN.....8
4.1 Architecture.....8
4.2 Technology Stack.....8
4.3 Request Flow and Security Design.....8
4.4 Authentication and OTP Design.....9
4.5 Match and Proctoring Flow.....9
5. DATABASE DESIGN.....11
5.1 Design Principles.....11
5.2 Entity Relationships.....11
5.3 Table Descriptions.....12
5.4 Seed Data.....12
5.5 Key Table Columns.....12
5.6 Integrity and Indexes.....13
6. MODULE DESCRIPTION.....14
7. IMPLEMENTATION.....19
8. USER INTERFACE.....24
9. TESTING.....29
9.1 Test Strategy and Environment.....29
9.2 Results and Observations.....29
9.3 Known Issues Found.....30
10. DEPLOYMENT AND PROJECT MANAGEMENT.....31
10.1 Deployment.....31
10.2 Team Structure.....31
10.3 Twelve-Week Plan.....31
11. CONCLUSION AND FUTURE SCOPE.....33
11.1 Conclusion.....33
11.2 Limitations.....33
11.3 Future Scope.....33
11.4 Delivered, Simulated and Planned.....33
11.5 Lessons Learnt.....34
REFERENCES.....35

LIST OF FIGURES

Figure 4.1: System architecture....8
Figure 4.2: Sign-up and OTP verification flow....9
Figure 4.3: Match, judging and proctoring flow....10
Figure 5.1: Entity relationship overview....11
Figure 7.1: Tournament money flow....20
Figure 8.1: Home page....24
Figure 8.2: Log in and sign-up....24
Figure 8.3: The Arena....25
Figure 8.4: Create a room....25
Figure 8.5: Host a tournament....26
Figure 8.6: Tournaments....26
Figure 8.7: Learning page....27
Figure 8.8: Profile....27
Figure 8.9: Host dashboard....28
Figure 8.10: Admin back office....28
Figure 10.1: Deployment pipeline....31
Figure 10.2: Twelve-week schedule....32

LIST OF ABBREVIATIONS

Short form	Meaning
API	Application Programming Interface
CSS	Cascading Style Sheets
CSE	Computer Science and Engineering
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JDBC	Java Database Connectivity
JSON	JavaScript Object Notation
JWT	JSON Web Token
OTP	One-Time Password
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
UI	User Interface

1. INTRODUCTION

1.1 Background

Competitive programming has become one of the most common ways for students to practise problem solving. Most platforms, however, are built for individual practice or for large public contests run by the platform itself. A college club, a coaching class or a group of friends who want to run their own contest, with their own rules, their own questions and sometimes a prize pool, has very few options. They usually fall back on shared documents, chat groups and manual scoring, which is slow, easy to dispute and open to cheating.

HackAgon is a web application that fills this gap. Any registered user can host an online coding competition in a few minutes, choose the format (solo, duo, squad or duel), pick questions from a shared bank, set the duration and invite players. A host who wants stakes can create a paid tournament: the prize pool is funded up front, entry fees are held in escrow, and the platform pays the prizes automatically according to a published split after deducting a flat 1% fee.

1.2 Problem Statement

There is no simple, trustworthy way for an ordinary host to run a fair, proctored and optionally paid coding contest. Hosts need a place to build the contest, players need confidence that the money is safe and that the result is honest, and organisers need tools to detect cheating without watching every screen. The project therefore aims to design and build a single web platform that covers the whole life of a contest: creation, registration, play, judging, proctoring, ranking, settlement and learning from mistakes.

1.3 Objectives

- Allow any verified user to create and host rooms in four formats: solo, duo, squad and duel.
- Judge submissions automatically against hidden test cases and rank players by points, time bonus and tie-break rules.
- Support free and paid tournaments with escrow, a published prize split, a 1% platform fee and a complete money ledger.
- Protect fairness through a camera gate, periodic webcam snapshots and a server-side tab-lock that cannot be bypassed by refreshing the page.
- Turn every failed attempt into a learning record, with a free library of books and an AI coach for hints and explanations.
- Verify every new account with a one-time password sent by e-mail, and keep all secrets on the server.

1.4 Scope of the Project

The delivered system is a complete web application with a Java and Spring Boot back end, a MySQL-compatible database hosted on TiDB Cloud, and fifteen HTML pages that make up the front end. It contains 76 REST endpoints across eight controller classes, a twenty-table schema, seed data of nineteen questions, eleven library books, seven achievements and seven platform rooms, and an automated smoke test that exercises the main flows.

Some parts are deliberately simulated in this build and are described honestly throughout the report. Payments use a simulated gateway that records charges as captured; a real gateway such as Razorpay is future scope. Submissions in JavaScript are really executed in a restricted sandbox, while the other four languages are judged by a deterministic simulation and are flagged as simulated; a hosted judge service such as Judge0 is future scope. One-time passwords are delivered by e-mail only; SMS delivery is future scope.

1.5 Organisation of the Report

Chapter 2 surveys existing approaches. Chapter 3 lists the requirements. Chapters 4 and 5 present the system and database design. Chapter 6 describes the ten modules and who built them, Chapter 7 explains the implementation, and Chapter 8 shows the user interface. Chapter 9 reports testing, Chapter 10 covers deployment and the twelve-week team plan, and Chapter 11 concludes with limitations and future scope. In this report the five team members are referred to as Hiten Sharma to Akshat Rathore according to the roles assigned in the team plan.

Chapter Summary

This chapter introduced HackAgon, a web platform on which any verified user can host a coding contest, join one, or pay for a tournament whose prize pool is held in escrow. It stated the problem, listed six objectives and fixed the scope of the delivered system. The table below repeats, at a glance, which parts of the system are working in this build and which are not.

Status	Features
Delivered	Accounts with e-mail OTP, rooms in four formats, judging, proctoring lock, escrow ledger, mistake book, library, achievements
Simulated	Payment gateway; judging of Java, Python, C++ and C
Future scope	Real payment gateway, SMS OTP, isolated judge service, 18+ rule for paid contests

The next chapter looks at how existing contest platforms, proctoring tools and escrow systems deal with the same problems.

2. LITERATURE SURVEY

2.1 Online Judges and Contest Platforms

Online judges such as LeetCode, HackerRank, Codeforces and CodeChef have shown that automatic evaluation against hidden test cases scales to millions of users. They compile or interpret submitted code inside isolated sandboxes, compare output with expected output, and enforce time and memory limits. Their contests are generally scheduled and owned by the platform or by approved setters, and the experience is built around public ranking lists.

Open-source judge engines such as Judge0 expose the same capability as an API, which lets smaller products embed code execution without building a sandbox from scratch. HackAgon follows the same principle: the judging logic sits behind a single module, so that the in-process execution used today can be replaced by a hosted judge service without changing the rest of the application.

2.2 Proctoring Approaches

Remote examinations use several signals to discourage cheating: forcing a full-screen view, detecting when the browser tab loses focus, blocking copy and paste, and capturing webcam images at intervals. Browser-only proctoring cannot be perfect, because a determined cheater can use a second device. Its value is in raising the effort and leaving evidence that a human can review.

A common weakness in simple implementations is that the lock lives only in the browser, so a refresh clears it. HackAgon stores the lock in the database against the player's seat in the room. After any violation the server itself refuses further submissions with HTTP status 423 until the host lifts the lock.

2.3 Money, Escrow and Fairness

Paid contests raise trust questions: will the prize exist, who decides the winner, and what happens in a dispute. Marketplaces solve this with escrow, where funds are held by a neutral party and released only when conditions are met. HackAgon applies the same idea. The host funds the pool before registration opens, entry fees are escrowed, a published split decides who receives what, and a dispute blocks payout until an administrator resolves it.

Contests that depend only on skill are treated differently from games of chance. HackAgon therefore has no lucky draws, spin wheels or random prizes. Ranking depends on solved questions, points and time, and prizes follow the published split.

2.4 Gap Analysis

Capability	Typical practice platform	HackAgon
Host own contest	Limited or approval based	Any verified user, in minutes
Formats	Mostly individual	Solo, duo, squad, duel
Prize pool	Platform decides	Host funds, escrow, published split
Proctoring	Varies	Camera gate, snapshots, server-side lock
Learning from mistakes	Submission history	Mistake book, free library, AI coach
Platform fee	Not applicable	Flat 1% of prize pool only

Chapter Summary

The survey showed that online judges are mature, that browser-only proctoring is useful but never perfect, and that paid contests need a neutral holder of money. No ordinary host, however, can combine a self-run contest, a proctored match and a guaranteed prize pool in one place. HackAgon was designed to fill this gap. The table links each finding of the survey to the design response taken in this project.

Finding from the survey	Design response in HackAgon
Online judges run code in isolated sandboxes and expose it as a service	Judging sits behind one module so the executor can be replaced by a hosted judge such as Judge0
Browser-only proctoring is lost on refresh and can be bypassed	The lock is stored in the database; after a violation the server refuses submissions with HTTP 423 until the host lifts it
Paid contests need a trusted holder of money	Escrow, a published prize split, a flat 1% fee on the prize pool and an append-only ledger; a dispute blocks payout
Skill-based contests differ from games of chance	No draws, spin wheels or random prizes; ranking depends only on solved questions, points and time

These choices are turned into formal requirements in Chapter 3.

3. REQUIREMENT ANALYSIS

3.1 Stakeholders

Four kinds of people use the system. A player joins rooms and tournaments, solves questions and tracks progress. A host creates rooms and tournaments, funds prize pools, monitors the match and unlocks players when a violation was a false alarm. An administrator manages the question bank and the library, reviews users and resolves disputes. The platform owner earns the 1% fee on prize pools and depends on the ledger for audit.

3.2 Functional Requirements

ID	Requirement
FR1	Sign up with name, username, e-mail, mobile, password and date of birth; verify the account with an e-mail OTP; log in with e-mail or username.
FR2	Create rooms in solo, duo, squad and duel formats with chosen languages, topics, questions, duration and visibility (public or private with an invite code).
FR3	Create tournaments with an entry fee, a funded prize pool, a prize preset (winner takes all, top 3, top 5 or custom) and optional proctoring.
FR4	Submit code for a question; judge it against hidden test cases; award points plus a time bonus; record failures as mistakes.
FR5	Camera gate, periodic snapshots, tab-lock on any violation, and host unlock.
FR6	Finalise a room, compute ranks with tie-breaks, settle the prize pool after a 1% fee, and write every money movement to a ledger.
FR7	Mistake book, library of free books, achievements and an AI coach for hints and explanations.
FR8	Administrator tools for the question bank, library review, users, disputes and money totals.

3.3 Non-Functional Requirements

- Security: passwords and OTPs are stored only as bcrypt hashes; sessions use signed JWTs valid for seven days; API keys and database passwords live in server environment variables and never reach the browser.
- Integrity of money: all amounts are stored as integer paise, never as floating point, and the ledger is append-only.
- Usability: pages work on a phone as well as a laptop, with a bottom navigation bar on small screens.
- Portability: the application runs on any host that supports Java and a MySQL-compatible database.
- Maintainability: routes, domain rules, judge, mail and AI access are separate modules.

3.4 Software and Hardware Requirements

Item	Requirement
Server runtime	Java 17 (JDK 17 or newer) with Maven
Framework and libraries	Spring Boot 3.2, Spring Security, JDBC, JJWT
Database	MySQL 8 compatible (TiDB Cloud in production; MariaDB 10.11 used for local testing)
Hosting	Render web service (free tier for demonstration)
Client	Any modern browser with webcam permission for proctored rooms
Developer machine	4 GB RAM, Git, VS Code

3.5 Feasibility Study

Technical feasibility is high because every component is a mature open-source tool and the whole system runs from one Spring Boot application. Economic feasibility is also high: the development stack is free, the demonstration deployment uses free tiers of Render and TiDB Cloud, and the only running costs in a real launch would be the e-mail or SMS provider and the payment gateway, which are covered by the 1% fee. Operational feasibility depends on keeping the platform skill-only and on obtaining proper payment-gateway approval before real money is handled; both are listed in the future scope.

3.6 Use Cases

Actor	Main use cases
Visitor	Read the landing page, sign up, verify the OTP, log in, reset a forgotten password.
Player	Browse the Arena, join a room or tournament, pay an entry fee, solve questions, view the scoreboard, review mistakes, read library books, ask the AI coach, view achievements.
Host	Create rooms and tournaments, fund the prize pool, start and finalise matches, view violations and snapshots, unlock or eliminate players, settle prizes.
Administrator	Manage questions and books, review users, resolve disputes, view money totals and the ledger.

3.7 Assumptions and Constraints

- Players have a modern browser, a stable connection and, for proctored rooms, a working webcam.
- The demonstration runs on free tiers of Render and TiDB Cloud, which may sleep or limit connections.
- Contests are skill-only. The platform does not offer draws, spins or any element of chance.

- Real money handling will only be switched on after a licensed gateway and the required approvals are in place.
- The first release targets English-language users in India, so amounts are in rupees stored as paise.

These assumptions shape later choices. For example, because the free database tier can limit the number of connections, the server uses a small pool of five connections, and because Render's free disk is temporary, the design treats snapshots as short-lived evidence rather than permanent records.

Chapter Summary

This chapter identified four kinds of user, listed the functional and non-functional requirements, set out the software and hardware needed, confirmed that the project is technically, economically and operationally feasible, and recorded the assumptions behind later decisions. The table maps each requirement area to the module of the team plan that delivers it.

Requirement area	Delivered by
Account creation and OTP verification	Module 1: Accounts and OTP
Hosting rooms and playing matches	Module 2: Rooms and matches; Module 8: Duels and teams
Judging and the question bank	Module 3: Code judge and question bank
Moderation, disputes and user control	Module 4: Admin and moderation
Entry fees, escrow, prizes and ledger	Module 5: Tournaments and money
Configuration, security and deployment	Module 6: Platform core and DevOps
Fair play and host monitoring	Module 7: Proctoring and host dashboard
Look, feel and responsiveness	Module 9: UI and design system
Learning from mistakes and AI help	Module 10: Learning, achievements and AI

Chapter 4 now shows how the system is put together to meet these requirements.

4. SYSTEM DESIGN

4.1 Architecture

HackAgon is a three-tier application served from a single origin. The browser loads static pages and scripts from the Spring Boot server and calls the JSON API under the path /api. The server talks to the database through a connection pool and to two external services: Gmail SMTP for one- time passwords and the Groq API for the AI coach.

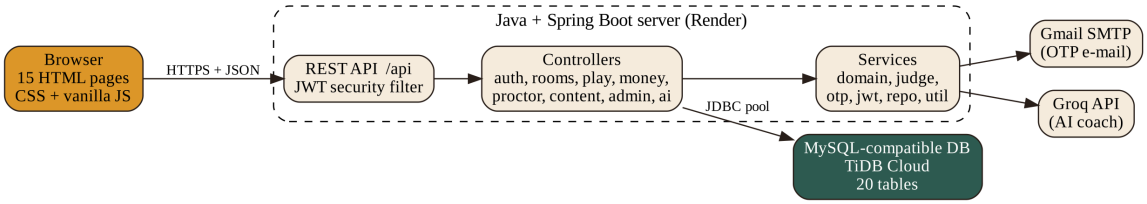


Figure 4.1: System architecture

4.2 Technology Stack

Layer	Technology	Reason
Front end	HTML, CSS, vanilla JavaScript	No build step; fast to host and easy to explain
Back end	Java 17, Spring Boot 3.2	Small, well known, single process
Database	MySQL via JDBC	Relational integrity for money and ranks
Security	BCrypt, JJWT	Hashed secrets, stateless sessions
Mail	Gmail SMTP or Brevo API	Free, simple, app-password based
AI	Groq chat completions	Fast inference for hints and explanations

The original product requirement document proposed Java, JSP and JDBC. The back end is built in Java 17 with Spring Boot 3.2, which keeps the Java and JDBC choice and replaces hand-written servlets with controller and service classes. The functional design did not change.

4.3 Request Flow and Security Design

Every protected request carries a bearer token. A security filter (SecurityConfig and JwtService) verifies the signature and expiry of the JWT, loads the user and rejects the call with HTTP 401 otherwise. Administrator routes add a second check on the user's admin flag. Validation errors return HTTP 422 with a field map so that the form can highlight the exact input. Errors are raised through a single HttpException class and converted to JSON by one handler (GlobalExceptionHandler), which keeps responses uniform.

Hidden test cases never leave the server. The bootstrap endpoint that feeds the browser with questions strips them out, and the browser receives only the visible examples. Snapshot uploads are limited to 400 KB, and the lock state is checked on the server for every submission.

4.4 Authentication and OTP Design

A new account is created in an unverified state. The server generates a six-digit code, stores only its bcrypt hash with a five-minute expiry and a limit of five attempts, and sends the code by e-mail. A new code may be requested after sixty seconds. Once the code is accepted the account becomes verified and a JWT is issued.

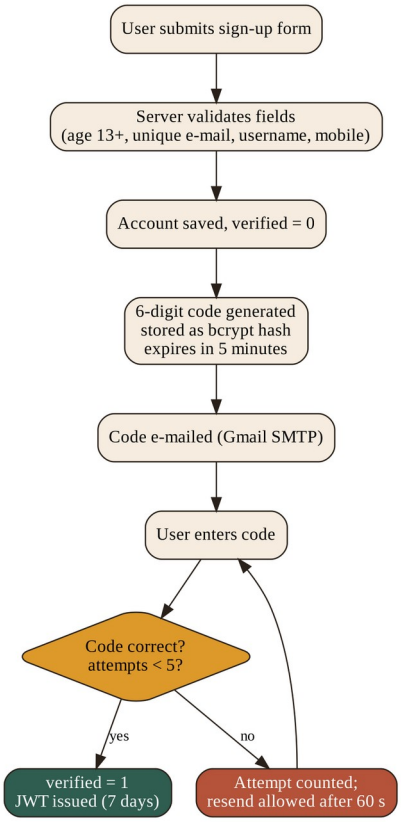


Figure 4.2: Sign-up and OTP verification flow

4.5 Match and Proctoring Flow

When a proctored match is running, the camera gate must pass before the editor opens. Each submission is judged and either scores or becomes a mistake. Any tab switch, window blur, paste, copy, cut, context menu, developer-tools use or exit from full screen is posted to the server, which locks the player and notifies the host.

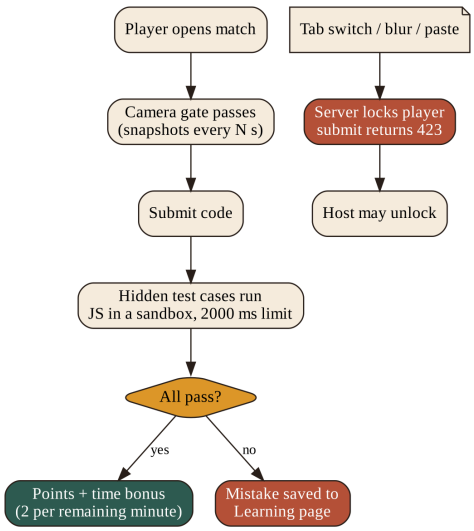


Figure 4.3: Match, judging and proctoring flow

Chapter Summary

The system is a three-tier web application served from one origin: the browser pages call a JSON API, the Spring Boot server applies the rules, and the database holds the records. The sign-up flow, the match flow and the proctoring flow shown in this chapter all follow one principle: the server, not the browser, decides. The table lists the main design decisions and the reason for each.

Decision	Reason
One origin for pages and API	No cross-origin set-up and a simpler deployment on a single Render service
JWT bearer token for sessions	Stateless sessions that the server checks on every protected request
OTP stored only as a hash	A leaked database does not reveal codes; expiry (five minutes) and an attempt limit (five) stop guessing
Hidden test cases never sent to the browser	Players see only the visible examples, so the cases that decide a verdict stay private
Proctoring lock kept on the server	Refreshing the page cannot clear a lock; every submission is checked against it

The data behind these flows is described in the next chapter.

5. DATABASE DESIGN

5.1 Design Principles

The schema has twenty tables. Identifiers are short prefixed strings such as u_ for users and room_ for rooms. All timestamps are stored as BIGINT milliseconds since the Unix epoch, which avoids time-zone problems between the server, the database and the browser. Flexible lists such as languages, topics, prize splits and statistics are stored in JSON columns. Money is stored in integer paise. The ledger table is only ever inserted into, never updated or deleted.

5.2 Entity Relationships

Users host rooms and sit in them through room_players. Submissions connect a user, a room and a question. Mistakes connect a user and a question. Violations and snapshots belong to a room and a user. Payments, payouts, ledger entries and disputes belong to a room. Prize presets, topics and languages are reference tables.

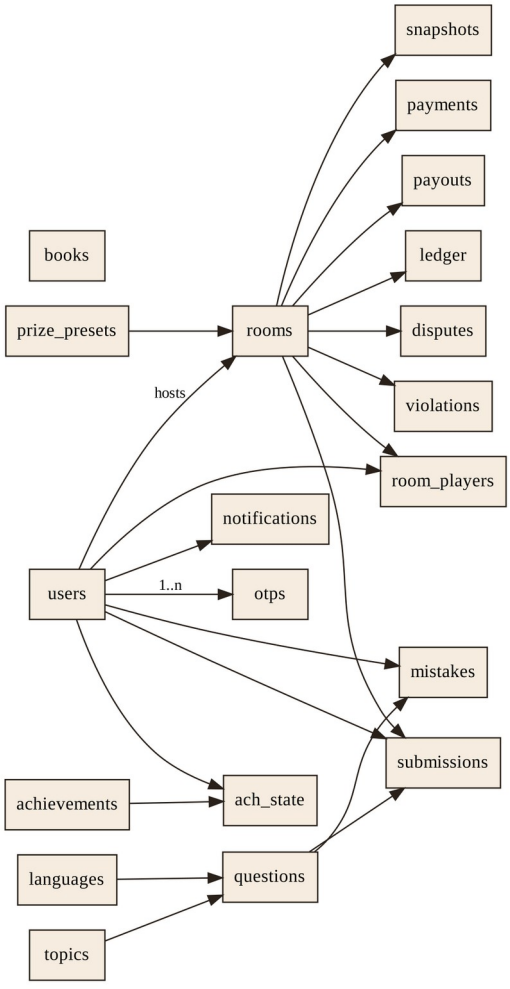


Figure 5.1: Entity relationship overview

5.3 Table Descriptions

Table	Purpose
users, otps	Accounts, profile, payout details, statistics; hashed one-time codes
topics, languages, prize_presets	Reference data used by forms
questions, books	Question bank with hidden cases; free library
achievements, ach_state	Achievement definitions; per-user progress
rooms, room_players	Contests and seats, scores, lock state
submissions, mistakes	Every attempt; failed questions to revisit
violations, snapshots	Proctoring events and webcam images
payments, payouts, ledger	Entry and pool charges; prizes; append-only money log
disputes, notifications	Payout disputes; in-app messages

5.4 Seed Data

A seed script fills the database with nineteen questions covering arrays, strings, linked lists, stacks, queues, trees, graphs, sorting, recursion and dynamic programming; eleven free books including Think Java, Automate the Boring Stuff with Python, Eloquent JavaScript, Open Data Structures and The Competitive Programmer's Handbook; seven achievements; five languages (Java, Python, C++, C, JavaScript); four prize presets; and seven platform rooms. Because the front end carries the same content in a seed file, the pages look populated even before any host creates a room.

5.5 Key Table Columns

Table	Important columns
users	id, name, username, email, mobile (all unique except name), password_hash (bcrypt), dob, verified, langs (JSON), payout (JSON), org (JSON), is_admin, stats (JSON), created_at
otps	id, email or mobile, code_hash, purpose (signup, login, forgot, payout, contact), expires_at, attempts, max_attempts, resend_at, verified
rooms	id, type (room, duel, tournament), host_id, format, size, langs, topics, question_ids, duration_min, visibility, invite_code, prize_pool_paise, entry_fee_paise, tournament_mode, snapshot_secs, status (draft, open, live, ended), start_time
room_players	room_id and user_id (composite key), score, solved (JSON), violations, camera, last_correct_at, locked, lock_reason, locked_at, eliminated
submissions	id, room_id, user_id, question_id, lang, code, passed, total, ok, time_ms, simulated, at
ledger	id, user_id (empty for platform entries), type (entry-fee, prize-fund, prize, refund, platform-fee), amount_paise, note, room_id, at

The composite key on room_players makes it impossible for a user to hold two seats in one room. Foreign keys with cascade delete remove a room's seats when the room is deleted. The simulated column on submissions records whether a verdict came from real execution, so that the report, the interface and any audit can tell the two apart.

5.6 Integrity and Indexes

- Unique keys on e-mail, username and mobile stop duplicate accounts at the database level, even if two sign-ups arrive at the same moment.
- Indexes on (room_id, user_id) in submissions and on (user_id, question_id) speed up scoreboards and the mistake book.
- Indexes on ledger (user_id, at) and (type) make personal statements and administrator totals fast.
- Transactions wrap money steps such as settlement, so a failure halfway cannot leave a prize paid without its ledger row.

Chapter Summary

The database has twenty tables. Money is stored as integer paise, the wallet ledger is append-only, and the structure of the tables stops the most common errors before they happen. The table below summarises the integrity measures described in this chapter.

Measure	Effect
Unique keys	E-mail, username and mobile cannot be duplicated, even when two sign-ups arrive together
Composite key on room_players	A user cannot hold two seats in the same room
Foreign keys with cascade	Deleting a room removes its seats
Indexes	Faster scoreboards, mistake book, personal statements and administrator totals
Transactions around money steps	A failed settlement cannot leave a prize paid without its ledger row

Chapter 6 explains which module reads and writes each group of tables.

6. MODULE DESCRIPTION

The application is divided into ten modules. Each of the five team members owns two modules, one larger and one smaller, so that the work is shared evenly. The owner named beside each module is the person who designed, built and tested it.

Module 1: Accounts and OTP (Hiten Sharma)

This module creates and protects user accounts. It covers sign-up, e-mail one-time passwords, login with e-mail or username, forgotten-password reset, profile editing and payout details.

- Validates name, username (3 to 20 letters, digits or underscore), e-mail, a ten-digit mobile number, a password of at least 8 characters and an age of 13 or more.
- Rejects duplicate e-mail, username and mobile with field-level messages.
- Hashes passwords with bcrypt and issues a JWT valid for seven days.
- Limits each OTP to five attempts, five minutes of life and one resend per minute.

Files: AuthController.java, OtpService.java, SecurityConfig.java, JwtService.java, signup.html, login.html, verify-otp.html. Tables: users, otps. Decision: OTP codes are stored only as bcrypt hashes, so a database leak does not reveal live codes.

Module 2: Rooms and Matches (Hiten Sharma)

This module is the heart of hosting. It lets a user create a room, publish it, let others join by browsing or by invite code, start the match and finalise the result.

- Room settings: format, capacity, languages, topics, questions, duration, visibility and optional per-question time.
- Private rooms receive a short invite code; public rooms appear in the Arena.
- The status moves from draft to open, live and ended; a proctored match cannot be left once it is live.
- Finalising a room fixes ranks using points and the tie-break rule.

Files: RoomsController.java, DomainService.java, arena.html, create-room.html, match.html. Tables: rooms, room_players. Decision: every seat in a room is a row in room_players, which also carries score, solved questions and lock state.

Module 3: Code Judge and Question Bank (Harsh Sharma)

This module evaluates submissions and serves questions. A submission is run against hidden test cases and the verdict is stored, scored and, when wrong, saved as a mistake.

- JavaScript runs in a restricted sandbox with a 2000 ms timeout.
- Java, Python, C++ and C use a deterministic simulation and the response carries simulated: true.

- The bootstrap endpoint sends visible examples only; hidden cases stay on the server.
- A practice endpoint lets a player retry a question outside any room.

Files: JudgeService.java, PlayController.java, ContentController.java. Tables: questions, submissions, mistakes. Decision: all judging sits behind one module so that Judge0 can replace it later without touching the routes.

Module 4: Admin and Moderation (Harsh Sharma)

This module is the back office. Administrators manage the question bank and the library, review accounts, resolve disputes and watch the money totals.

- Fourteen endpoints cover questions, books, users, disputes and room inspection.
- Questions can be created, edited and retired; library books carry a licence note and an approval status.
- A dispute blocks the payout of its room until an administrator decides.
- Addresses listed in ADMIN_EMAILS become administrators at sign-up.

Files: AdminController.java, admin.html, the make-admin script. Tables: questions, books, disputes, users. Decision: the admin gate checks the flag on the server for every call, not only in the page.

Module 5: Tournaments and Money (Hemanth Singh Tanwar)

This module handles everything that involves money: funding the pool, entry fees, the platform fee, prize splits, settlement, refunds and the ledger.

- All amounts are integer paise; the fee is Math.round(pool x 0.01).
- Prize presets: winner takes all, top 3 (50, 30, 20), top 5 (40, 25, 15, 10, 10) and custom.
- A tournament stays a draft until the pool is funded.
- Every movement is inserted into the ledger and never edited.

Files: MoneyController.java, IdUtil.java, create-tournament.html, tournaments.html, tournament- details.html. Tables: payments, payouts, ledger, prize_presets. Decision: the gateway is simulated and isolated, so a real gateway only replaces the capture step.

Module 6: Platform Core and DevOps (Hemanth Singh Tanwar)

This module keeps the system running. It starts the server, reads configuration, opens the database pool, creates the schema, seeds reference data, tests the flows and deploys.

- HackAgonApplication.java starts the server and mounts /api with a health check.
- The migration runner applies schema.sql and reports the number of tables created.
- The seed script loads questions, books, achievements, languages, topics, presets and platform rooms.

- The smoke-test script runs the end-to-end API checks; deployment goes from GitHub to Render with TiDB Cloud.

Files: HackAgonApplication.java, application.properties, DatabaseService.java, schema.sql. Decision: all secrets come from environment variables and are never committed.

Module 7: Proctoring and Host Dashboard (Arjun Suman)

This module protects fairness during proctored matches and gives the host a live view of the contest.

- Camera gate before the editor opens and periodic snapshots (at least every five seconds, default 25).
- Nine violation types are recorded: tab switch, blur, exit full screen, context menu, copy, cut, paste, developer tools and other.
- A violation locks the player on the server and notifies the host; submissions return HTTP 423.
- The host can view violations and snapshots and unlock or eliminate a player.

Files: ProctorController.java, host-dashboard.html. Tables: violations, snapshots, room_players. Decision: the lock is stored in the database, so refreshing or re-logging in cannot reopen a match.

Module 8: Duels and Teams (Arjun Suman)

This module adds the competitive formats beyond solo play: one-against-one duels, duo pairs and squads.

- Duel rooms are created from the duel page or the always-open practice duel.
- Duo and squad formats use a team size and count a question once for the team.
- Duel wins feed the Duelist achievement.
- Platform rooms (for example Daily Warm-up and Open Practice Duel) give new users something to join at once.

Files: duel.html, team and duel handling in DomainService.java and RoomsController.java. Tables: rooms, room_players. Decision: duels reuse the same room and scoring code instead of a separate engine.

Module 9: UI and Design System (Akshat Rathore)

This module gives the product its look and a consistent way of building screens. It defines colours, type, spacing and shared components used by all fifteen pages.

- Palette: cream background, terracotta, amber and forest green with a dark brown text colour.

- Type: Fraunces for headings, Karla for text, Kalam for handwritten accents and JetBrains Mono for code.
- Responsive layout with a bottom navigation bar on phones.
- api.js, store.js and app.js provide fetch handling, caching and the shared header.

Files: css/base.css, css/auth.css, css/components.css, css/theme.css, js/api.js, js/store.js, js/app.js. Decision: no framework and no build step, so the pages can be hosted by the same Spring Boot server.

Module 10: Learning, Achievements and AI (Akshat Rathore)

This module turns practice into progress. It keeps a record of failed questions, offers free books and rewards milestones, with an AI coach for help.

- The mistake book stores the last code, the failing case and a note on the right approach.
- The library lists eleven free books with their licence.
- Seven achievements are recomputed from stored statistics.
- The AI coach offers a hint and an explanation through the Groq API.

Files: learning.html, AiController.java, achievement logic in DomainService.java. Tables: mistakes, books, achievements, ach_state. Decision: the coach gives hints before answers, so it supports learning instead of replacing it.

6.11 Module Dependencies

The modules are loosely coupled, but a few depend on others. The table lists what each module needs and who agrees the interface.

Module	Depends on	Interface agreed by
Accounts and OTP	Platform core (mail, database)	Hiten Sharma and Hemanth Singh Tanwar
Rooms and matches	Accounts, question bank	Hiten Sharma and Harsh Sharma
Code judge and question bank	Rooms (submission context)	Harsh Sharma and Hiten Sharma
Admin and moderation	Accounts, money, question bank	Harsh Sharma and Hemanth Singh Tanwar
Tournaments and money	Rooms, accounts, admin (disputes)	Hemanth Singh Tanwar and Hiten Sharma
Platform core and DevOps	None; used by all	Hemanth Singh Tanwar
Proctoring and host dashboard	Rooms, tournaments	Arjun Suman and Hiten Sharma
Duels and teams	Rooms, judge	Arjun Suman and Harsh Sharma

Module	Depends on	Interface agreed by
UI and design system	All API modules	Akshat Rathore and all members
Learning, achievements and AI	Judge (mistakes), rooms (results)	Akshat Rathore and Harsh Sharma

6.12 Work Distribution Summary

Every member owns one three-part module and one two-part module in the original plan, so no one carries more than half again as much as another. Hiten Sharma owns the user-facing entry points, Harsh Sharma the judging and moderation tools, Hemanth Singh Tanwar the money and the platform, Arjun Suman the fairness features and the competitive formats, and Akshat Rathore the look of the product and its learning features. Shared files such as the schema, the server entry point and the API helper were edited only by Hemanth Singh Tanwar and Akshat Rathore, and other members asked for changes through pull requests.

6.13 A Paid Match Across the Modules

The following walk-through shows how the ten modules cooperate during one paid, proctored solo tournament.

- Accounts and OTP (Hiten Sharma): the host and the players sign up and verify their e-mail codes.
- Tournaments and money (Hemanth Singh Tanwar): the host creates the tournament as a draft and funds the pool; the room opens and the ledger records the escrow.
- Rooms and matches (Hiten Sharma): players find the tournament in the list and join; the entry fee is escrowed through the money module.
- Proctoring (Arjun Suman): each player passes the camera gate; snapshots start.
- Rooms and matches (Hiten Sharma): the host starts the room and the timer begins.
- Code judge (Harsh Sharma): each submission is judged against hidden cases, scored with the time bonus, and failures become mistakes.
- Proctoring (Arjun Suman): a tab switch locks one player, the host is notified and later unlocks the player.
- Rooms and matches (Hiten Sharma): the host finalises the room and ranks are fixed.
- Tournaments and money (Hemanth Singh Tanwar): the 1% fee is deducted, prizes are paid by the split and the ledger is updated.
- Learning (Akshat Rathore): achievements are recomputed and the mistake book is ready for each player.

7. IMPLEMENTATION

7.1 Project Structure and Code Size

The repository has a java-backend folder that holds the Spring Boot server and a set of front-end files at the top level. The server code is organised in a controller package with eight API controllers, a service package, a util package and a security configuration class. The front end has 3,340 lines of HTML in fifteen pages, 1,930 lines of JavaScript in five files and 700 lines of CSS in four files.

7.2 REST API

Controller	Endpoints	Main responsibility
AuthController	15	Sign-up, OTP, login, reset, profile, payout
RoomsController	12	Create, list, join, start, finalise rooms
AdminController	14	Questions, books, users, disputes
MoneyController	10	Fund, entry, settle, ledger, totals
ProctorController	8	Violations, locks, snapshots
ContentController	7	Bootstrap, questions, books, taxonomies
PlayController	6	Submit, practice, mistakes, notifications
AiController	4	Hint, explain, coach, status

7.3 Code Judge

A submission names the room, question, language and code. The judge wraps the user's code and runs each hidden case inside a restricted sandbox context with a 2000 ms limit, then compares the result with the expected value. The source code states clearly that the in-process sandbox is not a hardened security boundary. It is used here for the demonstration, and in a product build it is replaced by Judge0 or a similar isolated service. For Java, Python, C++ and C the same module returns a deterministic result and sets simulated to true, so the interface can show that the verdict is not from a real compiler.

7.4 Tournaments, Escrow and Settlement

A tournament starts as a draft. When the host funds the pool, the money is recorded as held in escrow and the room opens. Each player entry is recorded in the same way. When the room is finalised the settlement step computes the fee as the rounded one per cent of the pool, subtracts it, and divides the rest by the chosen preset: winner takes all, top three at 50, 30 and 20 per cent, top five at 40, 25, 15, 10 and 10 per cent, or a custom split. Every step writes a ledger row. An open dispute blocks settlement.

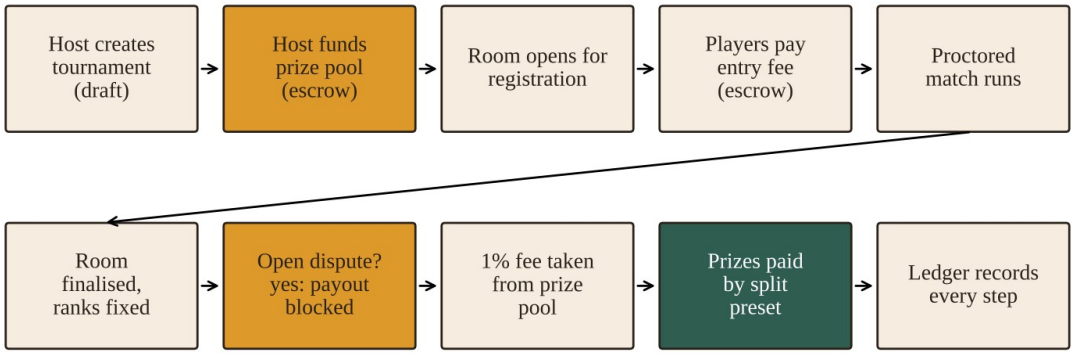


Figure 7.1: Tournament money flow

7.5 Proctoring

The match page asks for camera permission before the editor opens and uploads small JPEG snapshots at the interval chosen by the host, with a minimum of five seconds. Listeners watch for tab switches, blur, full-screen exit, context menu, copy, cut, paste and developer tools. Each event is posted to the server, stored in the violations table and followed by a lock on the player's seat. In the Render free tier, snapshot files live on local disk, which is erased on restart; durable object storage is future scope.

7.6 Scoring and Achievements

A correct solution earns the question's points plus a time bonus of two points for each whole minute remaining in the room. Ties in the final ranking are broken by the earlier time of the last correct submission. Achievements are recomputed from stored statistics: First Blood, Room Master, Perfectionist, Duelist, Tournament Conqueror, Clean Player and Scholar.

7.7 AI Coach

AiController.java sends a short prompt to the Groq chat completions API with the model openai/gpt-oss-120b. The hint endpoint nudges the player without giving the answer, and the explain endpoint describes why a failed attempt went wrong. If the API key is missing the route returns a clear message instead of failing.

7.8 Front-End Implementation

The pages share one stylesheet set and a small JavaScript layer. api.js wraps fetch, adds the bearer token stored under the key hackagon.token and turns error responses into readable messages. store.js caches data between pages, and app.js builds the header, bottom navigation and shared components.

7.9 Configuration

All configuration comes from environment variables that application.properties reads. The names below are the ones used on Render; the values are secrets and are never stored in the repository.

Variable	Purpose
PORT	Port the server listens on (8000 locally)
DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME	Database connection
DB_SSL	Turns on TLS, required by TiDB Cloud
JWT_SECRET, JWT_EXPIRES_IN	Signing key and lifetime of session tokens (7d)
EMAIL_HOST, EMAIL_PORT, EMAIL_SECURE, EMAIL_USER, EMAIL_APP_PASSWORD, EMAIL_FROM_NAME	Gmail SMTP settings for OTP mail
GROQ_API_KEY, GROQ_MODEL	AI coach access and model name
EXPOSE_OTP_IN_RESPONSE	Echoes OTP codes in API responses for testing; set to false in production
ADMIN_EMAILS	Comma-separated addresses that become administrators

7.10 Selected Code Excerpts

The following short excerpts, lightly shortened, show the rules that matter most. First, the platform fee, which is one per cent of the prize pool and is rounded to a whole paisa:

```
const platformFee = pool =>
  Math.round((Number(pool) || 0) * 0.01);
```

Second, the time bonus, which gives two points for every whole minute left in the room:

```
const timeBonus = Math.max(0,
  Math.round((room.durationMin * 60 - elapsedSecs) / 60) *
2);
const gained = q.points + timeBonus;
```

Third, the server-side lock that follows any proctoring violation:

```
UPDATE room_players
SET locked = 1, lock_reason = ?, locked_at = ?
WHERE room_id = ? AND user_id = ?
```

Fourth, the OTP limits, which are constants in OtpService.java:

```
const EXPIRY_MS = 5 * 60 * 1000;
const MAX_ATTEMPTS = 5;
const RESEND_MS = 60 * 1000;
```

7.11 Prize Presets

Preset	Split among ranked players
Winner takes all	Rank 1 receives 100% of the pool after the fee
Top 3	50%, 30% and 20%
Top 5	40%, 25%, 15%, 10% and 10%
Custom	Host-defined split that must add up to 100%

In the smoke test a pool of 50,000 paise with winner-takes-all produced a fee of 500 paise and a prize of 49,500 paise, and the ledger showed the escrowed fund, the entry fee, the fee and the prize.

7.12 Error Handling

The server answers with a consistent JSON shape. Missing or invalid tokens return 401, access to another user's room or an admin route returns 403, unknown ids return 404, invalid forms return 422 with a map of field messages, and a locked player's submission returns 423. Unexpected errors are logged on the server and returned as a short message without internal details.

7.13 Security Measures Implemented

- Passwords and OTP codes are stored only as bcrypt hashes.
- Sessions are signed JWTs checked on every protected request.
- Database queries use parameters, not string concatenation, which protects against SQL injection.
- Hidden test cases are removed from every response sent to the browser.
- Administrator routes check the admin flag on the server.
- Proctoring locks are stored in the database and enforced on submit with HTTP 423.
- Secrets live in environment variables; .env is ignored by Git.

7.14 Limits and Defaults

Item	Value
OTP expiry, attempts, resend gap	5 minutes, 5 attempts, 60 seconds
JWT lifetime	7 days
JavaScript judge timeout	2000 ms per run
Snapshot size limit	400 KB per image
Snapshot interval	Host-chosen, minimum 5 s, default 25 s
Database pool	5 connections
Platform fee	1% of the prize pool, rounded to a paisa
Time bonus	2 points per remaining minute

7.15 Sample API Interaction

A client logs in by posting an identifier (e-mail or username) and a password. The reply contains the JWT and the user record. Later calls send the token in the Authorization header.

```
POST /api/auth/login
{ "identifier": "host@example.com",
  "password": "********" }

200 OK
{ "ok": true, "token": "<jwt>", "user": { ... } }
```

Creating a funded tournament uses the rooms and money routes in turn. The body below is the one used by the smoke test (shortened):

```
POST /api/rooms
{ "name": "Smoke Test Tournament",
  "format": "Solo", "type": "tournament",
  "durationMin": 20, "prizePoolPaise": 50000,
  "entryFeePaise": 5000, "prizePreset": "winner-all" }

POST /api/money/rooms/:id/fund
POST /api/money/rooms/:id/entry
```

7.16 Health Check and Monitoring

The server exposes a health endpoint that reports whether the database is connected, which database version answered, whether the mail service is configured, whether OTP codes are being echoed and whether the AI coach is configured. Render uses the endpoint to decide whether the service is healthy, and the team uses it after every deployment to confirm that the environment variables were entered correctly. Server logs on Render show start-up messages, request errors and database connection problems.

Chapter Summary

The back end is organised into four packages under com.hackagon. The front end stays as plain HTML, CSS and JavaScript files served by the same Spring Boot application.

Package	Contents
config	SecurityConfig
controller	Auth, Rooms, Play, Content, Admin, Money, Proctor and Ai controllers, together with BaseController, HealthController and GlobalExceptionHandler
service	DatabaseService, DomainService, JudgeService, JwtService, OtpService and RepoService
util	HttpException and IdUtil

8. USER INTERFACE

This chapter shows the main screens of the running application, captured from the local test deployment with sample data. The design uses a warm cream background, terracotta and amber accents and a forest-green highlight for money and success.

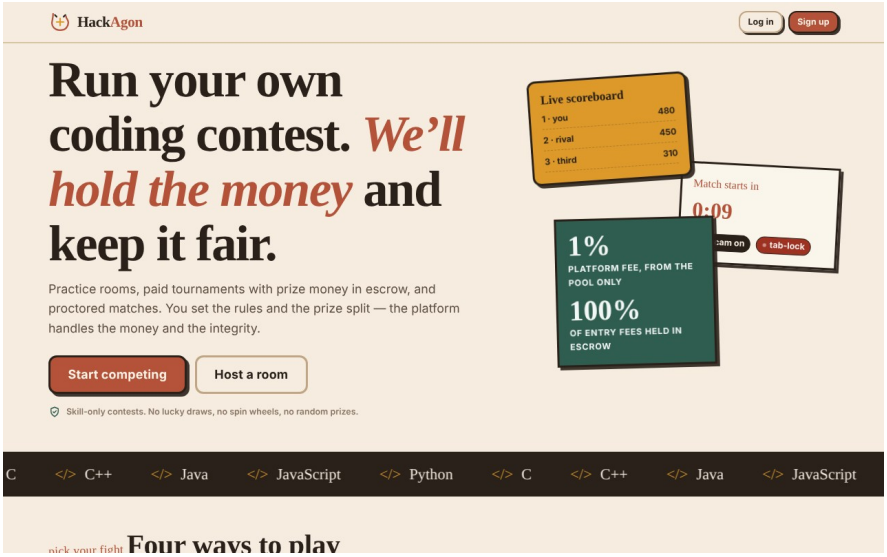


Figure 8.1: Home page

The landing page explains the idea in one screen: host your own contest, money held in escrow, a 1% platform fee and skill-only contests.

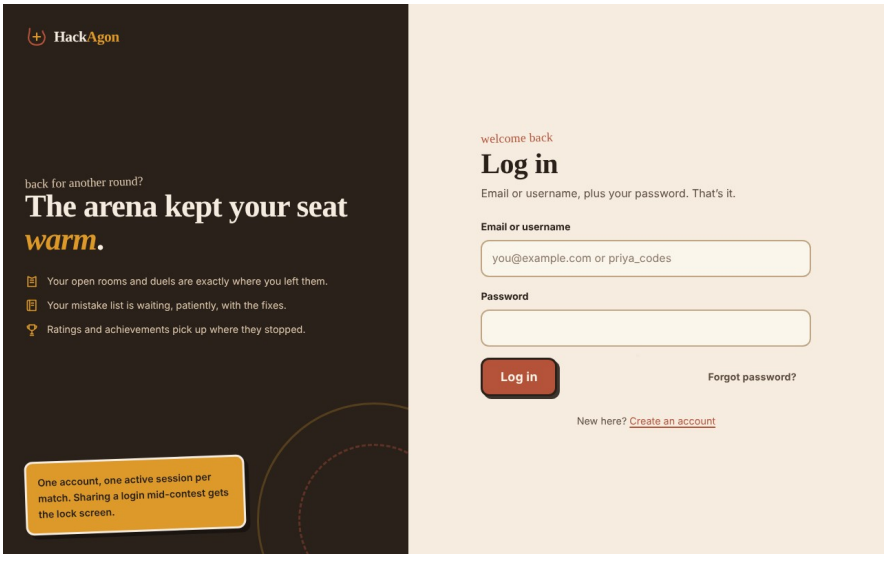


Figure 8.2: Log in and sign-up

Log in with e-mail or username. The sign-up form asks for name, username, date of birth, e-mail, mobile and password, then sends the code.

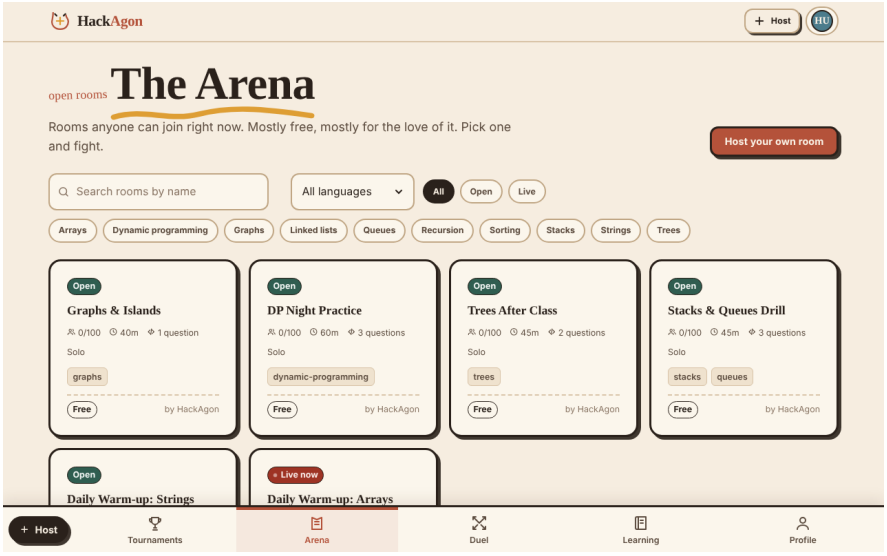


Figure 8.3: The Arena

Public rooms with search, language filter, topic chips and status badges for open and live rooms.

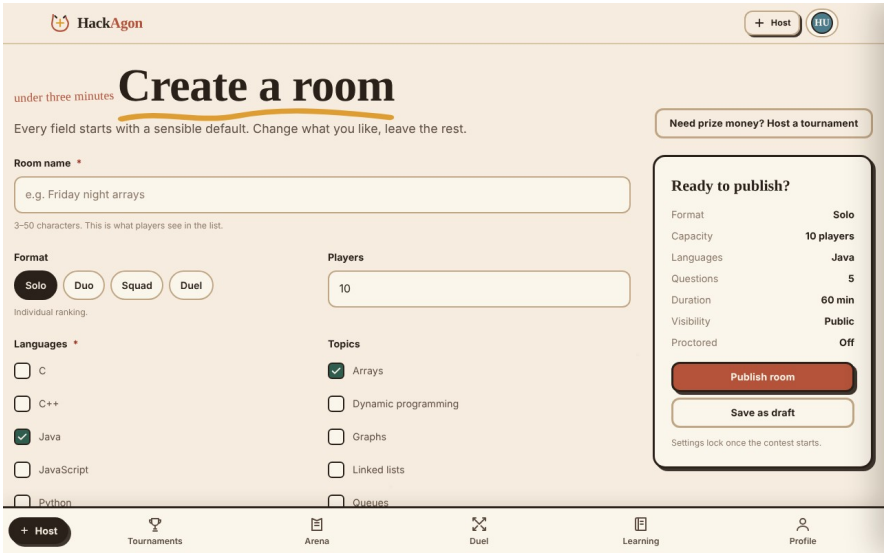


Figure 8.4: Create a room

A form with sensible defaults and a live summary card that shows format, capacity, languages and visibility.

Design Notes

- The Arena keeps open and live rooms apart through status badges, so a player sees at once which rooms can still be joined.
- The create-room form never starts empty: every field has a default, and the summary card shows what the host is about to publish.

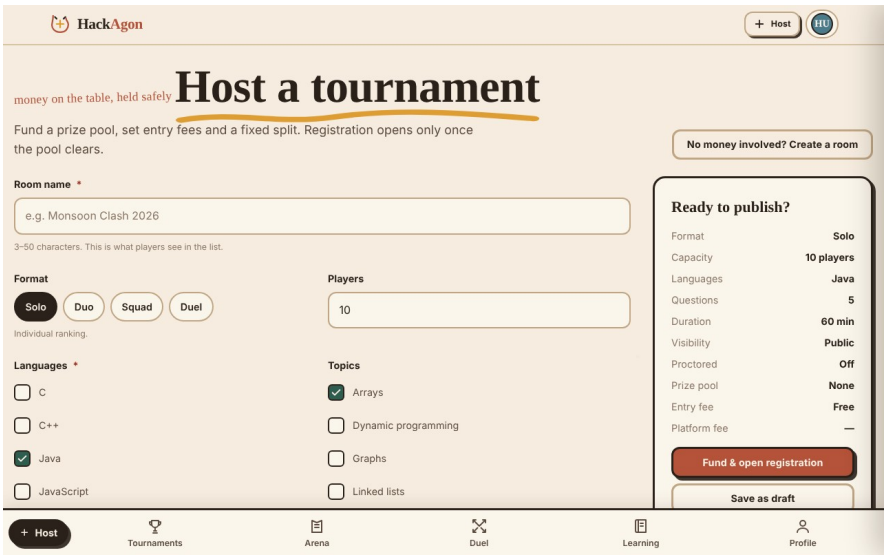


Figure 8.5: Host a tournament

The same form with entry fee, prize pool, prize preset and proctoring, with a button that funds and opens the room.

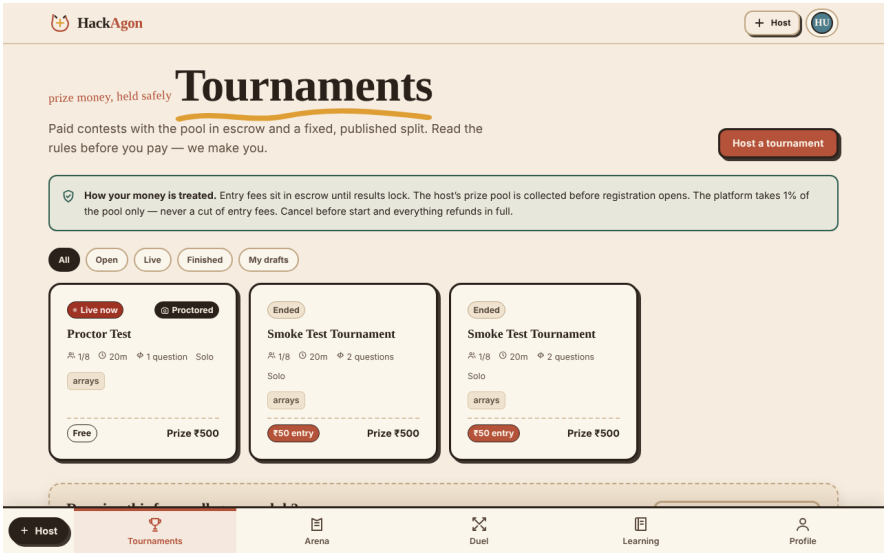


Figure 8.6: Tournaments

Paid contests with a notice that explains escrow and the 1% fee, and filters for open, live, finished and drafts.

Design Notes

- A tournament cannot open until the host funds it; the form shows entry fee, prize pool and preset before any money moves.
- The Tournaments page states the escrow rule and the 1% fee in plain words before a player pays.

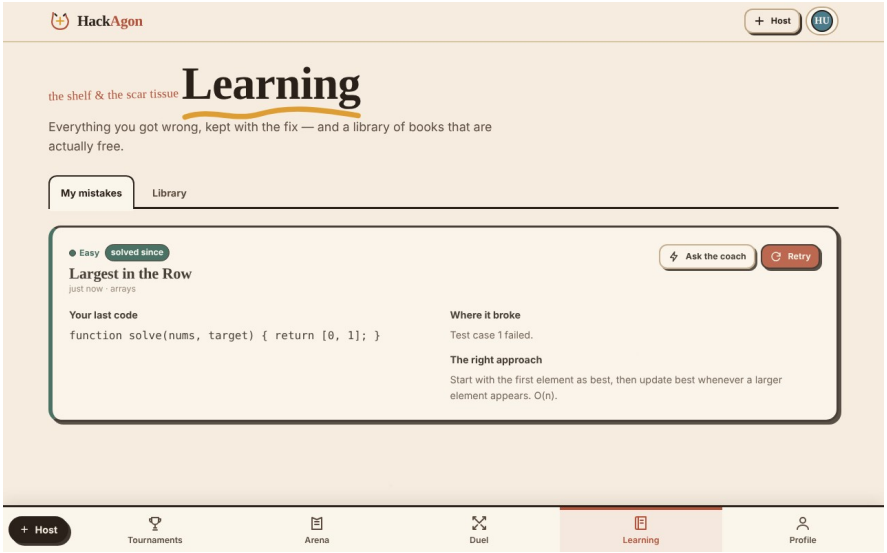


Figure 8.7: Learning page

My mistakes keeps the last code, the failed test and the right approach; the Library tab lists free books.

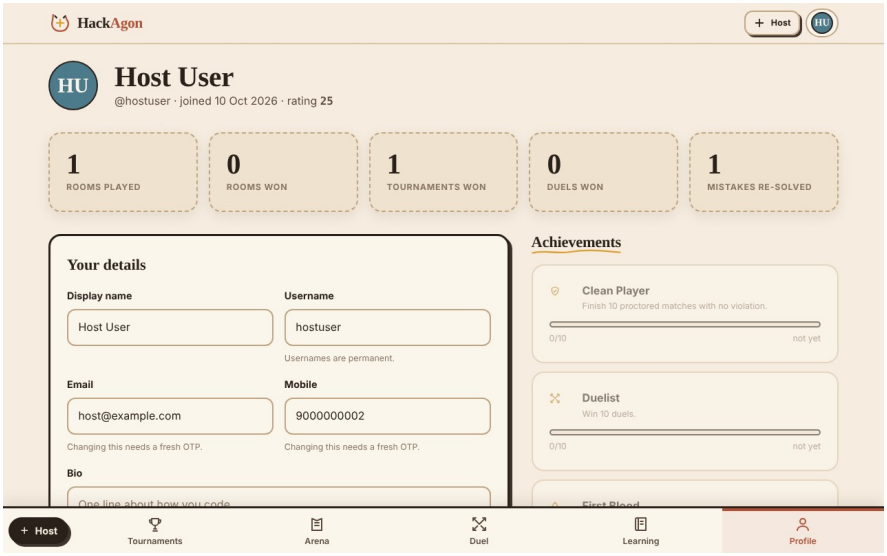


Figure 8.8: Profile

Statistics, editable details and achievement progress bars.

Design Notes

- Every mistake card keeps the last code, the failing case and the right approach, so a retry starts from evidence rather than memory.
- Profile statistics and achievement bars grow only from real activity; a new account shows zero everywhere.

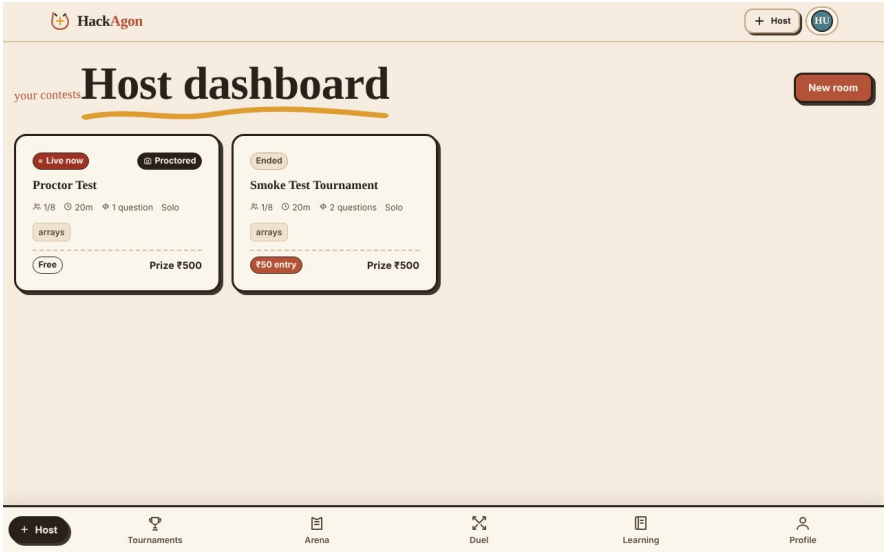


Figure 8.9: Host dashboard

Cards for each hosted contest with status, proctoring badge, entry fee and prize.

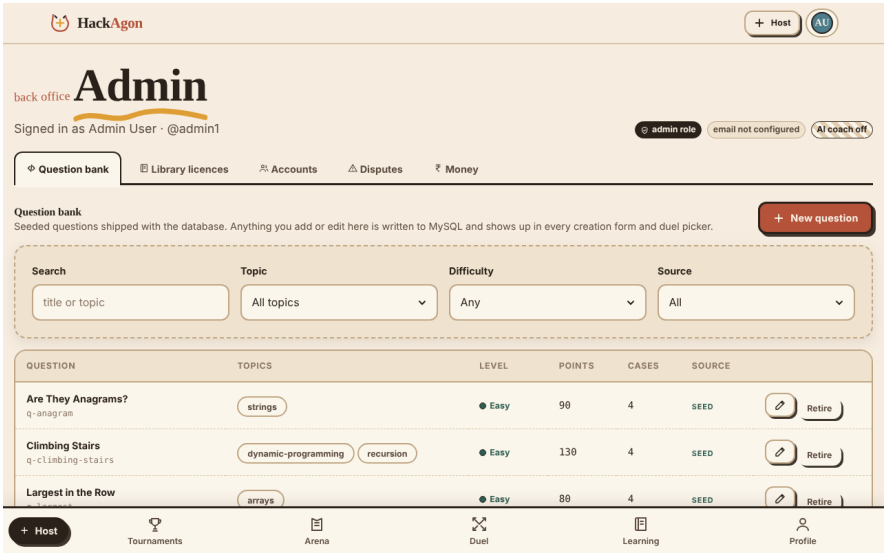


Figure 8.10: Admin back office

Tabs for the question bank, library licences, accounts, disputes and money.

Design Notes

- The host dashboard puts status, proctoring badge, entry fee and prize on one card, so a host can see each contest without opening it.
- The admin back office is protected by a server-side administrator check, so the screens are only usable by administrators.

9. TESTING

9.1 Test Strategy and Environment

Testing combined an automated API smoke test with manual walk-throughs of every page. The smoke test logs in, creates a funded tournament, plays through a match, locks and unlocks a player, settles the prize pool and reads the ledger. It was first run against a local copy of the server connected to MariaDB 10.11, built before the move to Java, and the same checks are to be repeated on the Java 17 build. The production stack uses TiDB Cloud, which speaks the same MySQL protocol.

No.	Test case	Result
1	Login returns a JWT (administrator account)	Pass
2	GET /auth/me returns profile with 7 achievements	Pass
3	Bootstrap data hides hidden test cases (19 questions, 7 rooms)	Pass
4	Administrator gate admits an admin and lists users	Pass
5	Create tournament: unfunded stays a draft	Pass
6	Host is auto-joined; funding opens the room; entry fee escrowed	Pass
7	Host starts the room	Pass
8	Correct JavaScript passes 4 of 4 hidden cases and scores +140	Pass
9	Wrong answer fails and is booked in the mistake book	Pass
10	Non-JavaScript verdict is flagged as simulated	Pass
11	Settlement: fee is 500 paise (1% of 50,000); winner receives 49,500	Pass
12	Ledger records escrow, entry fee, fee and prize	Pass
13	Administrator money totals endpoint	Pass
14	Proctored room: a violation locks the player on the server	Pass
15	Locked player's submission returns HTTP 423	Pass
16	Host unlock clears the lock; violation is logged	Pass
17	AI coach hint and explanation (needs Groq key)	Not run

9.2 Results and Observations

In the administrator run of the smoke test 17 checks passed and 4 did not. Two of the four are the AI-coach checks, which returned the message that the coach is not configured because no Groq key exists in the test environment; they remain untested here and should be repeated on the deployed server. The other two proctoring checks failed only because the smoke script creates a room without turning proctoring on, so the server correctly reported that the room is not proctored. A separate test script created a proctored room and the three proctoring checks (cases

14 to 16) passed. A run with a non-administrator account correctly failed the administrator checks, which confirms that the gate works.

Manual checks covered sign-up with an invalid age, duplicate e-mail and short password, all of which returned field-level errors, and the layout of each page on a desktop and a phone-sized window. No payment gateway, SMS service or non-JavaScript compiler was tested because none is connected in this build.

9.3 Known Issues Found

- The smoke script should be updated to pass tournamentMode when it tests proctoring.
- Snapshots are stored on local disk, so they disappear when the free Render instance restarts.
- The age check in code is 13 or older; the product rule of 18 or older for paid contests is not yet enforced.
- The free Render instance sleeps after inactivity, so the first request after a pause is slow.

Chapter Summary

Testing combined an automated API smoke test with manual walk-throughs of every page. Every listed case passed except the AI coach case, which was not run because no Groq key existed in the test environment. The smoke test was run on the earlier build of the server connected to MariaDB 10.11, and the same checks are to be repeated on the Java 17 build.

Area	Result
Automated smoke test	Login, funded tournament, match, lock and unlock, settlement and ledger
Manual checks	Invalid age, duplicate e-mail and short password return field-level errors; every page checked on desktop and phone-sized windows
Not tested	Real payment gateway, SMS service, compilers for languages other than JavaScript
To be repeated	AI coach checks on the deployed server; proctoring checks with a proctored room

The deployment and the project plan that produced these results are described in the next chapter.

10. DEPLOYMENT AND PROJECT MANAGEMENT

10.1 Deployment

The code lives in the GitHub repository ASR-C/HACK-AGON. The database is a TiDB Cloud serverless cluster, prepared once by running the schema and seed data against it. Render builds the service from the repository with Maven and starts the packaged Spring Boot jar. Configuration is supplied through environment variables on Render: the database host, port, user, password, name and TLS flag, the JWT secret, the Gmail address and app password, the Groq key and model, the administrator e-mail list, and the flag that decides whether OTP codes are echoed in API responses (set to false once real e-mail works).

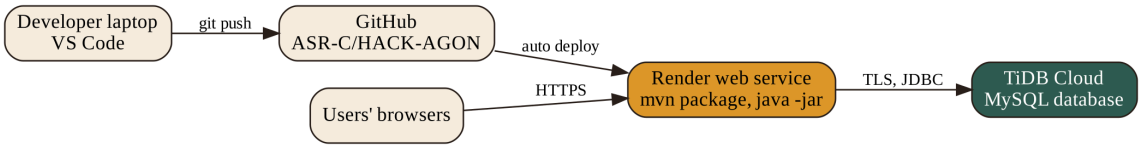


Figure 10.1: Deployment pipeline

10.2 Team Structure

Member	Modules owned
Hiten Sharma	Accounts and OTP; Rooms and matches
Harsh Sharma	Code judge and question bank; Admin and moderation
Hemanth Singh Tanwar	Tournaments and money; Platform core and DevOps
Arjun Suman	Proctoring and host dashboard; Duels and teams
Akshat Rathore	UI and design system; Learning, achievements and AI

10.3 Twelve-Week Plan

The project ran from 6 July 2026 to 6 October 2026. The first twelve weeks (6 July to 27 September) were organised as weekly work packages with three milestones: a working solo match on 2 August, a full paid test tournament on 30 August and the first deployed release on 27 September. The last days, 28 September to 6 October, were used for the final review, the report and submission.

Deployment Checklist

- Enter every environment variable exactly as it is shown in the dashboard of the database and mail provider.
- Make sure the database user and name match the cluster, then run the schema and seed data once.
- Open the health endpoint after each deployment and confirm that the database, mail and AI settings report as connected.
- Switch the OTP echo flag to false as soon as real e-mail delivery works.

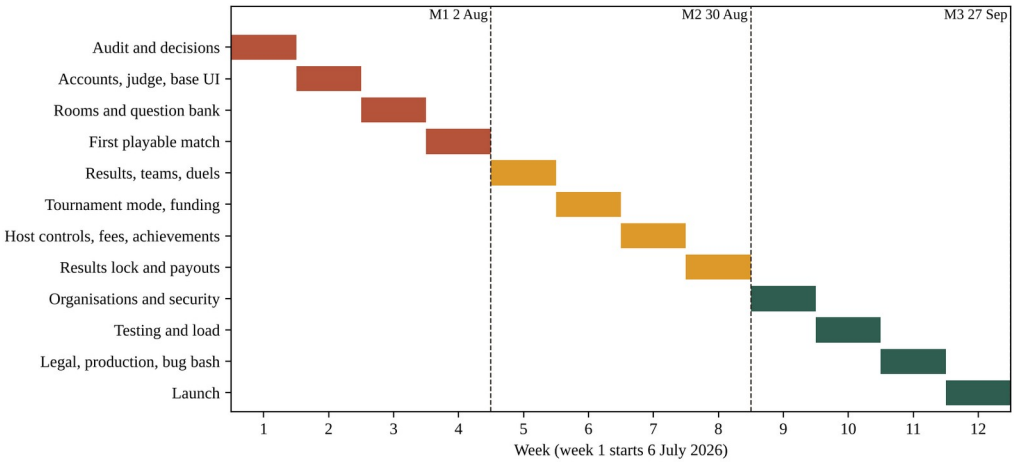


Figure 10.2: Twelve-week schedule

Week	Dates	Focus of the week
1	6 to 12 Jul	Audit and decisions: accounts audit, judge spike, database and repo set-up, proctoring spike, design direction
2	13 to 19 Jul	Accounts, judge and base UI: OTP and profile, judge end to end, schema V1, components
3	20 to 26 Jul	Rooms and question bank: room creation, admin question screens, security middleware
4	27 Jul to 2 Aug	First playable match, judge hardening, match page. Milestone 1 on 2 Aug
5	3 to 9 Aug	Results, teams and duels; money tables and ledger; mistake book
6	10 to 16 Aug	Tournament mode and funding; webcam and tab lock; library page
7	17 to 23 Aug	Host controls, entry fees, disputes, achievements
8	24 to 30 Aug	Result locking and payouts; proctoring trial. Milestone 2 on 30 Aug
9	31 Aug to 6 Sep	Organisations, security review, AI hints and design pass
10	7 to 13 Sep	End-to-end and load tests, proctoring edge cases, accessibility
11	14 to 20 Sep	Legal pages, production set-up, bug bash, landing page
12	21 to 27 Sep	Launch and first release on Render. Milestone 3 on 27 Sep
Final	28 Sep to 6 Oct	Final review, report and submission

Some items in this schedule, such as SMS OTP, a live payment gateway and an external judge service, go beyond the build described in this report and are listed in the future scope.

11. CONCLUSION AND FUTURE SCOPE

11.1 Conclusion

HackAgon shows that a small team can build a complete contest platform with ordinary web tools. A host can create a room or a paid tournament, players can join and solve questions, the server judges and ranks them, proctoring leaves evidence and enforces a lock, and the money moves through escrow to a published split with a clear ledger. The learning side turns each failure into a retry list, a free book and an AI explanation.

Just as important, the project forced honest engineering. The report separates what really works from what is simulated: JavaScript judging is real, other languages are simulated; the payment gateway is simulated; e-mail OTP is real while SMS is planned. This separation shows exactly which parts must be replaced before real money and real users are involved.

11.2 Limitations

- The in-process judge is not a hardened security boundary and must not be exposed to untrusted users; it is replaced by an isolated service in the product build.
- Languages other than JavaScript are judged by simulation.
- The payment gateway is simulated, so no real money moves.
- OTP is sent only by e-mail through a personal Gmail account with daily sending limits.
- Snapshots use local disk and are lost on restart in the free hosting tier.
- Browser proctoring cannot detect a second device.

11.3 Future Scope

- Integrate Judge0 for real, isolated execution in Java, Python, C++ and C.
- Integrate a licensed payment gateway such as Razorpay with UPI payouts and proper verification of hosts.
- Add SMS OTP through a provider such as MSG91.
- Enforce the 18+ rule for paid contests and add identity checks for large prize pools.
- Move snapshots to object storage and add automated review of faces and screens.
- Add a native mobile app, live scoreboards through WebSockets and team leagues.

11.4 Delivered, Simulated and Planned

Feature	Status in this build
Accounts, e-mail OTP, JWT sessions	Delivered
Rooms, duels, teams, invite codes	Delivered
JavaScript judging with hidden cases	Delivered (real execution)
Java, Python, C++, C judging	Simulated, flagged in the response

Feature	Status in this build
Escrow, 1% fee, prize split, ledger	Delivered with a simulated gateway
Camera gate, snapshots, tab-lock	Delivered; snapshots on temporary disk
Mistake book, library, achievements	Delivered
AI coach	Delivered; needs a Groq key to run
SMS OTP, real payment gateway, 18+ check	Planned (future scope)

11.5 Lessons Learnt

The team learnt that clear module ownership prevents most conflicts: only a few shared files, such as the schema and the server entry point, touch every module, so changes to them were agreed in advance. Keeping money in integer paise and writing every movement to a ledger made the settlement logic easy to test. Storing the proctoring lock on the server, rather than in the browser, removed a whole class of bypasses.

Deployment taught practical lessons too: environment variables must be copied exactly, a database user and name must match the cluster, free hosting sleeps and loses local files, and a smoke test that can be run against any environment is worth the time it takes to write.

Closing Remarks

HackAgon began as a question: can an ordinary student club host a fair, proctored and optionally paid contest without trusting a stranger with the money? The twelve weeks of work between 6 July and 27 September 2026, followed by the final review until 6 October 2026, gave a working answer. A host can create a contest, players can compete under a server-side lock, and prize money follows a published split through an escrow ledger.

The report has tried to be as clear about what is missing as about what works. The simulated payment gateway, the simulated judging of four languages and the absence of SMS delivery are not hidden; they are the first items of future work. We hope the design, the code and this report are useful to the students who build on them next.

Hiten Sharma, Harsh Sharma, Hemanth Singh Tanwar, Arjun Suman and Akshat Rathore

REFERENCES

[1] Wasik, S., Antczak, M., Badura, J., Laskowski, A., Sternal, T. A Survey on Online Judge Systems and Their Applications. ACM Computing Surveys, vol. 51, no. 1, Article 3, 2018. arXiv:1710.05913.

[2] Rahman, M. M., Watanobe, Y., Shirafuji, A., Hamada, M. Exploring Automated Code Evaluation Systems and Resources for Code Analysis: A Comprehensive Survey. arXiv:2307.08705, 2023.

[3] Khmelevsky, Y., Chidlow, K. Students Programming Competitions as an Educational Tool and a Motivational Incentive to Students. arXiv:2105.15136, 2021.

[4] Heintz, F., Farnqvist, T. Pedagogical Experiences of Competitive Elements in an Algorithms Course. Conference on Teaching and Learning (Proceedings), Lund University, 2013.

[5] Mohammed, H. M., Ali, Q. I. Cheating Prevention in E-proctoring Systems Using Secure Exam Browsers: A Case Study. Jurnal Ilmiah Teknik Elektro Komputer dan Informatika (JITEKI), vol. 8, no. 4, pp. 634-648, 2022. doi:10.26555/jiteki.v8i4.25094.

[6] Atoum, Y., Chen, L., Liu, A. X., Hsu, S. D. H., Liu, X. Automated Online Exam Proctoring. IEEE Transactions on Multimedia, vol. 19, no. 7, pp. 1609-1624, 2017. doi:10.1109/TMM.2017.2656064.

[7] Dawson, P. Five Ways to Hack and Cheat with Bring-Your-Own-Device Electronic Examinations. British Journal of Educational Technology, vol. 47, no. 4, pp. 592-600, 2016. doi:10.1111/bjet.12246.

[8] Nigam, A., Pasricha, R., Singh, T., Churi, P. A Systematic Review on AI-based Proctoring Systems: Past, Present and Future. Education and Information Technologies, vol. 26, no. 5, pp. 6421-6445, 2021. doi:10.1007/s10639-021-10597-x.

[9] Witkowski, J., Seuken, S., Parkes, D. C. Incentive-Compatible Escrow Mechanisms. Proceedings of the 25th AAAI Conference on Artificial Intelligence (AAAI-11), 2011.

[10] Abhishek, K., Roshan, S., Kumar, A., Ranjan, R. A Comprehensive Study on Two-Factor Authentication with One Time Passwords. In Computer Networks and Communications (NetCom 2012), Lecture Notes in Electrical Engineering, vol. 131, Springer, pp. 405-415, 2013. doi:10.1007/978-1-4614-6154-8_40.

[11] Muszynski, T., Koziel, G. A Security Analysis of Authentication and Authorization Implemented in Web Applications Based on the REST Architecture. Journal of Computer Sciences Institute, vol. 16, pp. 252-260, 2020.

[12] Yang, J., Wang, E., Chen, J., Wang, Q., Zhang, Y., Duan, H., Xie, W., Wang, B. Token Time Bomb: Evaluating JWT Implementations for Vulnerability Discovery. Network and Distributed System Security (NDSS) Symposium, 2026.

[13] Jones, M., Bradley, J., Sakimura, N. JSON Web Token (JWT), RFC 7519, IETF, 2015.

[14] Provos, N., Mazieres, D. A Future-Adaptable Password Scheme. USENIX Annual Technical Conference, 1999.

Technical Documentation Consulted

[15] Oracle. Java SE 17 Documentation. https://docs.oracle.com/en/java/javase/17/

[16] VMware. Spring Boot 3.2 Reference Documentation. https://docs.spring.io/spring-boot/docs/3.2.0/reference/html/

[17] jwtk. JJWT: Java JWT library. https://github.com/jwtk/jjwt

[18] Oracle. MySQL Connector/J Developer Guide. https://dev.mysql.com/doc/connector-j/en/

[19] PingCAP. TiDB Cloud Documentation. https://docs.pingcap.com/tidbcloud

[20] Brevo. Transactional Email API. https://developers.brevo.com

[21] Groq. API Documentation. https://console.groq.com/docs

[22] Render. Web Services Documentation. https://render.com/docs

Tools Used in the Project

Tool	Use
Java 17 (JDK) and Maven	Language and build tool of the back end
Spring Boot 3.2	Web, security and JDBC support for the server
VS Code	Editor used by the team
Git and GitHub	Version control, pull-request review and deployment source
Render	Hosting of the web service
TiDB Cloud and MariaDB	Production database and local test database
Brevo and Gmail SMTP	E-mail delivery of one-time passwords
Groq API	Hints and explanations of the AI coach